逻辑回归实验

从模型训练到 Agentic Programming

教学补充手册

覆盖原手册 §1.1–§1.6，并新增 Agent / Prompt / Context / Harness 单元

01

数据与模型

从 47 列到 44 维，从 logit 到概率

02

训练与评估

BCELoss、L1 正则、混淆矩阵与 ROC

03

Agentic 工作流

Agent、Prompt、Context 与 Harness

版本 1.0 · 2026-10 · 建议 2–3 学时

配套：教学 PPT · logreg_lab 完成版 · submission 提交包

00 写在前面

这本手册解决什么问题，怎么用

这本手册是《逻辑回归-实验手册》的教学补充版。它保留原手册的实验目的、原理、数据清洗、模型实现、训练评估、环境与提交、10 分评分细则，并补上原手册没有展开的三块内容：MinMaxScaler 与张量形状、测试评估指标、以及 Agent / Prompt / Context / Harness。

写作对象是第一次做机器学习实验的本科生。代码与数据来自工作区 logreg_lab；原手册的 data_utils 保持原版不改；新增内容尽量用“本实验里的具体对象”举例，而不是泛泛讲概念。

表 0-1 三条阅读路径

阅读路径	建议读什么	读完能做什么
课前 20 分钟	00 速览 + 01 实验目的 + 02 原理	知道任务、指标和 10 分评分点
课中边做边查	03–11 数据、模型、训练、评估	能补全 notebook 并跑出指标
课后扩展	12 提交 + 13–16 Agent 单元	能写 AGENTS.md、验收清单与使用声明

本手册的边界

它是“补充 + 重写”，不是替代原手册。原手册的代码骨架、data_utils、10 分评分细则仍然是最终依据；当两者表述不一致时，以原手册和实际数据文件为准，本手册会明确标注差异。

0.1 符号与约定

表 0-2 常用记号

记号 / 术语	含义
x, y	一条样本的特征向量与标签；$y \in \{0, 1\}$
w, b	权重与偏置；本实验 w 形状 (44, 1)、b 形状 (1,)
z / logit	线性分数（式 2-1）
p / ŷ	sigmoid(z)（式 2-2）
BCE	二元交叉熵；逻辑回归的负对数似然
L1 / L2	权重绝对值之和 / 权重平方和（式 9-1 / 9-2）
Harness	让 Agent 能看、能做、能记、能验收的软件系统

00 实验速览

先记住 6 个数字和 1 张评分表

表 0-3 实验一页纸

项目	内容
任务	根据贷款申请人的信息，预测 isDefault（1 = 违约）
数据	10,000 条记录；训练 8000 / 验证 1000 / 测试 1000；47 列
输入	清洗后 44 维：40 个原始数值 + 4 个编码列
模型	单层逻辑回归：线性分数 + sigmoid（式 2-1 / 2-2）
训练	BCELoss + L1 正则；AdamW，lr=0.005，100 epochs，batch=64
评估	Accuracy、Precision、Recall、F1、ROC-AUC、混淆矩阵
交付	可执行 logic.ipynb + 最后 markdown 反思 + 学号_姓名_logic.zip

0.2 先记住 6 个数字

表 0-4 关键数字

数字	含义	为什么重要
47 → 44	原始列 → 入模特征	减法链：−id −标签 −subGrade
8000 / 1000 / 1000	train / val / test	验证集用来选模型，测试集只最后看一次
50.06%	训练集正样本比例	50% 是准确率基线
5595	训练集缺失单元	19 列有缺失；n11 最多 624
0.6550	本实验验证集 Accuracy	与手册示例 0.655 对齐
0.6350 / 0.6897	测试集 Accuracy / ROC-AUC	中等信号；不要过度解读 1% 差异

一句话

先把“能不能跑通”与“结果可不可信”分开：前者靠形状、dtype、日志；后者靠验证集、混淆矩阵、AUC 与人工复核。

00 本手册结构

16 节 + 2 个附录

节	标题	你会得到什么
01	实验目的、产出与验收	3 个目的、交付物、10 分评分表
02	逻辑回归原理	从回归到分类、sigmoid、MLE → BCE
03	数据与 44 维	47 列、44 维、划分、口径冲突
04	数据清洗	日期、grade、employmentLength 的转换
05	缺失值	先删标签、再填特征、顺序与 dtype
06	缩放与张量	MinMax fit/transform、float32、形状
07	Dataset / DataLoader	两个魔术方法、batch 数、独立 generator
08	模型实现	参数、sigmoid、forward、45 个参数
09	损失与正则化	BCE、L1/L2、norm_weight、AdamW 默认值
10	训练	五步循环、验证、模板里的三个问题
11	评估与结果	混淆矩阵、F1、AUC、曲线与结果解读
12	环境、提交与评分	命令、VSCode、命名、评分对照
13	Agent 协作准备	AGENTS.md、目录边界、上下文卫生
14	用 Agent 定位与验证 bug	5 个真实坑、最小复现、验证证据
15	用 Agent 做记录与验收	10 分清单、ACCEPTANCE、verify_submission
16	边界、引用与学术诚信	三条红线、使用声明、引用规范

附录 A：命令与数字速查；附录 B：三份可直接改写的 Prompt / 模板。

01 实验目的、产出与验收

先把“做什么、交什么、怎么给分”对齐

1.1 三个实验目的

01 用 pandas 完成数据读取、缺失值处理、类型转换与基础清洗，并理解 Dataset / DataLoader 的作用。

02 理解逻辑回归：线性分数、sigmoid、最大似然与交叉熵，能解释每个公式在代码里的位置。

03 构建一个能处理结构化表格的二分类模型，完成训练、测试评估，并能用指标解释结果。

这份实验不是“从零写一个框架”，而是接管一套已经给了数据、工具函数和 notebook 骨架的项目：补全 TODO、跑通、验证、写清楚。它同时是后面 Agent / Harness 单元的可验收样本。

1.2 交付物

表 1-1 交付物清单

交付物	要求	对应评分
logic.ipynb	从头到尾可执行；无残留 TODO / 裸 pass	全部评分项
测试集指标	Accuracy，建议加 Precision / Recall / F1 / AUC	训练与评估 1 分
最后 markdown 反思	收获、难度、建议；必须在最后一格	训练与评估 1 分
学号_姓名_logic.zip	按手册 §1.5.4 命名，提交到 BB	提交规范

1.3 官方 10 分评分细则

评分块	分值	细则	证据
环境与数据	3	环境 0.5 / 读三集 0.5 / 清洗 1 / Dataset+DataLoader 1	env、shape、0 NaN、batch 数
模型	4	初始化 1 / sigmoid 1 / forward 1 / L1-L2 1	w/b、输出 (N,1)、normnize
训练与评估	3	损失 1 / 测试评估 1 / 反思 1	BCE、指标、最后一格 markdown
合计	10		

> **先看评分，再写代码**
>
> 评分项不是“知识点列表”，而是“可观察的证据”。每写一个函数，问自己：如果老师只打开 notebook，能不能在 30 秒内找到证据?

1.4 和 Agent 单元的关系

这个实验的 10 分细则、可运行 notebook 和 verify_submission.py，构成了一个“可验收任务”。后面的 Agent / Prompt / Harness 单元会反复回到这个任务：Agent 能补代码、能跑测试，但它不会自动替你判断“数据划分是否合理”“指标口径是否对”“反思是不是自己写的”。

02 逻辑回归原理

从线性分数到概率，再到损失

2.1 为什么不是直接做线性回归?

我们要预测“是否违约”，输出只有 0 或 1。如果直接拿线性回归拟合，输出可能是 -0.3 或 1.7，既没有概率含义，也无法设定阈值。逻辑回归的做法是：先用线性模型算一个无界的分数，再用 sigmoid 把它压到 (0, 1)。

表 2-1 回归与分类的区别

任务	输出	能不能当概率
预测房价	连续值，如 320.5 万	有数量含义，但不是概率
预测违约	0/1 类别	需要 sigmoid 映射到概率
逻辑回归	概率 p（式 2-2）	是概率；阈值 0.5 只是决策规则

2.2 logit 与 sigmoid

线性分数（logit）把特征加权求和；w 是每个特征的权重，b 是偏置。

$$z = w^T x + b$$

式 2-1 线性分数（logit）

再用 sigmoid 把 z 映射到 (0, 1)：

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

式 2-2 sigmoid

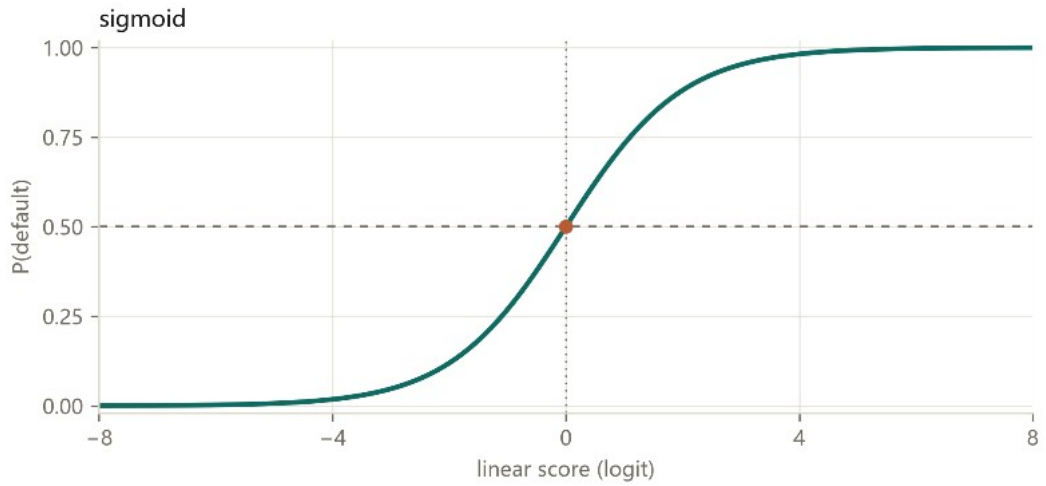


图 2-1 sigmoid 把任意分数压到 (0,1)；z=0 时 p=0.5。

手算三个点：z 为 0 时概率 0.5；±2 时约 0.88 / 0.12。p 是模型估计的违约概率，不是硬标签；把 p 变成 0/1 需要阈值。

2.3 最大似然 → 交叉熵

把每个样本看成一次伯努利试验：

$$P(y = 1 \mid x) = p, \quad P(y = 0 \mid x) = 1 - p$$

式 2-3 伯努利概率

假设样本独立，似然是各样本概率的连乘：

$$L = \prod_i p_i^{y_i}(1 - p_i)^{1 - y_i}$$

式 2-4 似然

取负对数，连乘变连加，得到 BCE：

$$BCE = -\left[y \log p + (1 - y) \log (1 - p)\right]$$

式 2-5 二元交叉熵

表 2-2 四个手算例子

真实 y	预测 p	BCE	直觉
1	0.90	0.105	自信且正确，惩罚小
1	0.10	2.303	自信且错误，惩罚大
0	0.10	0.105	自信且正确，惩罚小
0	0.90	2.303	自信且错误，惩罚大

为什么不是平方误差

BCE 来自伯努利分布的负对数似然；它对“自信且错误”的惩罚会急剧增大。平方误差也能优化，但它对应高斯噪声假设，不是二分类的概率模型。

2.4 四个术语

表 2-3 术语对照

术语	本实验中的含义
训练集 / 验证集 / 测试集	8000 拟合参数 / 1000 选模型 / 1000 最后评估
样本	一条贷款记录，记为 $x^{(i)}$
标签	isDefault，0 或 1，记为 $y^{(i)}$
特征	入模的 44 列，记为 x 的各个维度

动手

在 notebook 里手算一条样本：取 w、b 和一条 x，算 z、p、BCE；再和 torch 的输出对比。这能帮你确认“公式—代码”是对齐的。

03 数据与 44 维

47 列、三个划分与一张列账本

3.1 任务与数据

数据来自某信贷平台的贷款记录。每条记录 47 列，标签 isDefault：1 表示违约。文件被拆成 train_sub.csv（8000）、val_sub.csv（1000）、test_sub.csv（1000）。

表 3-1 三个划分

数据划分	行数	正样本	负样本	正样本比例
train	8000	4005	3995	50.06%
val	1000	505	495	50.50%
test	1000	490	510	49.00%

训练集接近 1:1，所以 50% 是准确率基线；测试集也只有 1000 条，准确率标准误约 1.5 个百分点。

3.2 47 列怎么变成 44 维

表 3-2 列账本

减法	列	原因
47	原始 CSV	含 id、标签和所有特征
− 1	id	唯一标识，对预测无贡献
− 1	isDefault	标签，不能当输入
− 1	subGrade	与 grade 冗余，data_utils 中删除
= 44	输入特征	40 个原始数值 + 4 个编码列

44 维的构成：40 个原始数值列（33 连续 + 9 离散，含 policyCode 常量）加 4 个编码列（grade、employmentLength、issueDate、earliesCreditLine）。自检：train_input.size(1) == 44。

3.3 原始材料的三个口径冲突

表 3-3 口径差异

冲突	原始说法	以什么为准
划分	PPT 说 8000 训练 + 2000 测试	实际三个 CSV：8000/1000/1000
grade 等级	PPT/手册写 A–D	实际 A–G 共 7 级
特征计数	42 数值 + 5 类别 + 15 匿名	以列账本为准：47 − id − 标签 − subGrade = 44

课堂提醒

不要背“42/5/15”这三个数；背“47 − 1 − 1 − 1 = 44”。前者是数据字典的近似说法，后者是你能用代码验证的账本。

3.4 数据体检

表 3-4 数据体检

项目	train	val	test
行 × 列	8000 × 47	1000 × 47	1000 × 47
缺失单元	5595	734	968
有缺失的列	19	17	16
id 唯一	是	是	是
常量列	policyCode	policyCode	policyCode

训练集缺失最多的列：n11（624）、employmentLength（518）、n0–n14 多数 325、revolUtil（11）、dti / pubRecBankruptcies（各 3）。

04 数据清洗

把字符串、日期和类别变成模型能吃的数字

utils.py 里的 data_utils 已经封装了四步：date_init → grade → employmentLength → astype(float)。本实验不修改 utils.py；你要理解它做了什么，并能在 notebook 里调用 pipeline()。

4.1 日期：两个列，两种单位

表 4-1 日期处理

原始列	格式	转换	范围（train）
issueDate	yyyy-mm-dd	距本划分最早日期的天数	0–4018 天
earliesCreditLine	MMM-YYYY，如 Mar-2007	距最早月份的月数	0–695 月

代码 4-1 日期转换（utils.py 节选）

```
df['issueDate'] = pd.to_datetime(df['issueDate'], format='%Y-%m-%d')
min_issue = df['issueDate'].min()
df['issueDate'] = (df['issueDate'] - min_issue).dt.days

df['earliesCreditLine'] = pd.to_datetime(df['earliesCreditLine'], format='%b-%Y')
min_date = df['earliesCreditLine'].min()
df['earliesCreditLine'] = ((df['earliesCreditLine'].dt.year - min_date.year) * 12
                           + (df['earliesCreditLine'].dt.month - min_date.month))
```

> **方法学提醒**
>
> data_utils 对每个划分各自取最早日期；严格做法是只在 train 上拟合日期基准，再 transform val/test。本实验为对齐原手册保留原版；你可以在 worklog 里指出这一点，但不要擅自改数据划分。

4.2 grade、subGrade 与 employmentLength

表 4-2 grade 与就业年限

列	规则	例子
grade	ord(首字母) − ord('A') + 1	A→1, B→2, …, G→7
subGrade	删除（与 grade 冗余）	B1、B2 等 35 个值不进入模型
employmentLength	'< 1'→1；'10+'→12；n years→n+1	'3 years' → 4

employmentLength 的 n+1 映射不是等距的；grade 是序数编码，但 A–G 之间的“距离”也不一定相等。这些都属于课程给定的基线处理，先按规则跑通，再在报告里讨论局限。

4.3 完整流水线

代码 4-2 pipeline 的顺序

```
class data_utils:
    def pipeline(self):
        self.date_init()
        self.grade()
        self.employmentLength()
        self.df = self.df.astype(float)
        return self.df
```

顺序很重要：先在原始字符串上做日期/类别转换，最后 astype(float)。如果先转 float，字符串列会直接报错；如果先填充均值，object 列也无法求均值。

> **动手**
>
> 打开 train_sub.csv，取前 3 行，手算 issueDate、grade、employmentLength 的转换结果，再和 data_utils.pipeline() 的输出对比。

05 缺失值

标签不能猜，特征可以填

训练集有 5595 个缺失单元，19 列有缺失；val 734、test 968。缺失集中在匿名计数特征 n0–n14、employmentLength 和 n11。处理顺序是：先删标签缺失行，再对输入特征按列均值填充。

5.1 先删标签缺失

代码 5-1 删除标签缺失行

```
def filter_NA(df):
    return df.dropna(subset=['isDefault']).reset_index(drop=True)
```

标签是监督学习的“答案”。答案缺失的行无法计算损失，也不能从特征可靠恢复，所以只能删。本数据标签没有缺失，因此行数不变；但代码仍要写，保证对缺失标签的鲁棒性。

5.2 再填特征均值

代码 5-2 按列均值填充

```
def fill_mean(df):
    return df.fillna(df.mean(numeric_only=True))
```

均值填充是基线：简单、可解释、不会引入额外模型。也可以用中位数、众数或模型填补，但要在报告里说明。注意 fill_mean 必须在 pipeline 转 float 之后调用，否则 object 列无法求均值。

5.3 顺序为什么不能反

表 5-1 处理顺序

顺序	结果
先 fill_mean 再 filter_NA	标签也被均值填上，缺失标签行被当成正常样本
先 filter_NA 再 fill_mean	标签缺失行删除；输入缺失用均值填补（正确）
先 astype(float) 再 fill_mean	字符串列转换失败或语义丢失

填充后要做验收：三份数据的 isnull().sum().sum() 都应为 0。不要只看 train。

> **方法学提醒**
>
> fill_mean 对每个 split 各自求均值；严格做法是只用 train 的均值，再应用到 val/test。原手册这样写，本实验为对齐评分保留；课堂上应指出这个差异。

> **动手**
>
> 打印三份数据的缺失数；故意把 fill_mean 放在 filter_NA 之前跑一次，观察行数和标签分布的变化。

06 缩放与张量

MinMax 只在 train 上 fit

年收入、余额、贷款金额与利率的量级相差 3–4 个数量级。不缩放时，梯度会被大量级特征主导，收敛慢甚至不稳定。本实验用 MinMaxScaler 把每个特征缩放到 [0, 1]。

6.1 fit 与 transform 的区别

表 6-1 MinMax 的三个动作

操作	在谁身上做	学到什么
fit	train	每列的最小值、最大值
transform	train / val / test	用 train 的 min/max 做线性变换
fit_transform	train	fit + transform 的简写

> **防泄漏**
>
> 只在 train 上 fit；如果对全量数据 fit，测试集的 min/max 会进入训练，造成信息泄漏。这是实验报告里最常被问的点之一。

代码 6-1 缩放

```
scaler = MinMaxScaler()
train_input = scaler.fit_transform(train_input.values)
val_input = scaler.transform(val_input.values)
test_input = scaler.transform(test_input.values)
```

6.2 转成张量

表 6-2 张量形状

对象	形状	dtype	说明
X_train	(8000, 44)	float32	特征矩阵
y_train	(8000,)	float32	0/1 标签，1-D
X_val / y_val	(1000, 44) / (1000,)	float32	验证
X_test / y_test	(1000, 44) / (1000,)	float32	测试

utils.py 里还有一个被注释掉的 z_score()。本实验用 MinMaxScaler，不启用 z_score；不要在同一个 notebook 里混用两套缩放，否则指标不可比。

> **动手**
>
> 打印 train_input.dtype、shape、min/max；再检查 val/test 的 min/max 是否可能超出 [0,1]（超出是正常的 transform 结果，不是 bug）。

07 Dataset 与 DataLoader

Dataset 管一条，DataLoader 管一批

PyTorch 的数据读取分两层：Dataset 定义“第 i 条样本是什么、一共多少条”；DataLoader 负责切 batch、打乱、并行加载。

7.1 MyDataset：两个魔术方法

代码 7-1 MyDataset

```
class MyDataset(Dataset):
    def __init__(self, *tensors):
        assert all(t.size(0) == tensors[0].size(0) for t in tensors)
        self.tensor = tensors

    def __len__(self):
        return self.tensor[0].size(0)

    def __getitem__(self, index):
        return tuple(t[index] for t in self.tensor)
```

assert 让形状错误在构造 Dataset 时就暴露，而不是训练到一半才报错。__getitem__ 返回 (x_i, y_i)：x 形状 (44,)，y 是标量。

7.2 DataLoader 与 batch 数

代码 7-2 DataLoader

```
train_loader = DataLoader(MyDataset(X_train, y_train),
                          batch_size=64, shuffle=True)
val_loader = DataLoader(MyDataset(X_val, y_val),
                        batch_size=64, shuffle=False)
```

表 7-1 batch 数（向上取整）

数据	样本数	batch 数	末批大小
train	8000	125	64
val	1000	16	40
test	1000	16	40

7.3 只打乱 train，并用独立 generator

shuffle=True 只给训练集；验证/测试保持顺序，指标才可对齐。DataLoader 即使 shuffle=False，每次 iter() 也会消耗全局 RNG；如果 notebook 和脚本的验证循环次数不同，训练集的打乱顺序会变，结果就无法逐位复现。

代码 7-3 独立 generator

```
generator = torch.Generator()
generator.manual_seed(42)
train_loader = DataLoader(..., shuffle=True, generator=generator)
```

真实 bug

本实验完成版曾出现 notebook 与脚本 Accuracy 0.6310 vs 0.6320 的差异，根因就是验证集迭代消耗 RNG。换成独立 generator 后两者指标逐位一致。

动手

打印 len(train_loader)、每个 batch 的 xb.shape；再跑两次同 seed 的训练，确认指标一致。

08 模型实现

一个类，45 个参数

模型只有一层：w (44,1) 和 b (1,)，共 45 个参数。它很小，所以效果不好时先怀疑数据和特征，而不是“模型不够大”。

8.1 类骨架

代码 8-1 模型骨架

```
class LogisticRegression(nn.Module):
    def __init__(self, input_dim, norm='l2'):
        super().__init__()
        assert norm in ['l1', 'l2']
        self.w = nn.Parameter(torch.randn(input_dim, 1))
        self.b = nn.Parameter(torch.randn(1))
        self.norm = norm

    def sigmoid(self, x):
        return torch.sigmoid(x)

    def forward(self, x):
        return self.sigmoid(x @ self.w + self.b)
```

8.2 三个关键点

表 8-1 三个关键点

关键点	正确做法	常见错误
形状	p (N,1)，y (N,)；BCE 前 flatten	直接 BCELoss(p, y) → 广播成 (N,N)
数值	clamp 概率到 [1e-7, 1-1e-7]	sigmoid 饱和 → log(0) → NaN
正则	L1 / L2 范数（式 9-1 / 9-2）	把 norm 当 loss 直接相加但忘了 λ

8.3 L1 / L2 与 criterion

代码 8-2 正则与损失

```
def l1_norm(self):
    return torch.abs(self.w).sum()

def l2_norm(self):
    return (self.w ** 2).sum()

def criterion(p, y):
    p = torch.clamp(p.view(-1), 1e-7, 1 - 1e-7)
    return nn.BCELoss()(p, y.view(-1))
```

normnize() 按 self.norm 分派；总损失为：

$$L_{total} = BCE + \lambda R(w)$$

式 8-1 总损失

其中 norm_weight 控制正则强度。注意 BCELoss 期望概率在 (0,1)，所以 forward 返回 sigmoid 后的概率，而不是 logit。

> **自检数字**
>
> w: (44,1)，b: (1,)，参数量 45；forward 输入 (N,44)，输出 (N,1)；损失是一个标量，不能是 (N,N)。

09 损失与正则化

BCE、L1/L2 与 λ 的尺度

9.1 BCE 的三个细节

BCE 是逻辑回归的负对数似然。代码里有三个容易忽略的细节：

01 预测和标签先 flatten 到 (N,)；否则 (N,1) 与 (N,) 会广播。

02 概率 clamp 到 [1e-7, 1-1e-7]；sigmoid 饱和后 log(0) 会变成 -inf。

03 标签用 float32，不要用 long；BCELoss 需要与概率同 dtype。

代码 9-1 criterion 的核心

```
loss_fn = nn.BCELoss()
p = torch.clamp(p.view(-1), 1e-7, 1 - 1e-7)
loss = loss_fn(p, y.view(-1))
```

9.2 L1 / L2 的区别

两种正则项的写法：

$$R_{L1}(w) = \lambda \sum_j |w_j|$$

式 9-1 L1 正则

$$R_{L2}(w) = \lambda \sum_j w_j^2$$

式 9-2 L2 正则

L1 让很多权重变成 0；L2 让权重整体变小但不为 0。

表 9-1 L1 与 L2 的效果

正则	效果	本实验
L1	稀疏：很多权重变 0	34/44 个权重≈0
L2	收缩：权重变小但不为 0	更平滑
无正则	权重更大，可能过拟合	测试 AUC 0.6921

本实验用 L1 + norm_weight=0.005。L1 范数从 31.5 降到 3.61，说明权重被明显压缩；测试 AUC 与 L2/无正则接近（0.6897 / 0.6914 / 0.6921），所以 L1 的主要收益是稀疏与可解释，不是更高的分数。

9.3 AdamW 的默认 weight_decay

模板写的是 AdamW(model.parameters(), lr=0.005)，没有显式设置 weight_decay；AdamW 默认 weight_decay=0.01，相当于额外加了一层 L2 衰减。它不是错，但你要知道它存在，否则调正则时会混淆。

实验对照

L1、L2、无正则三组测试 AUC 的差异远小于 1000 条测试样本的抽样误差。不要根据 0.001 的差异宣称某个正则“更好”。

动手

把 norm_weight 改成 0、0.001、0.05，各跑 20 epochs，记录 L1 范数、验证 AUC 和零权重数量。

10 训练

五步循环与一次验证

10.1 配置

表 10-1 训练配置

超参数	值	说明
optimizer	AdamW	自适应学习率 + 解耦权重衰减
lr	0.005	手册配置
epochs	100	本数据约 10 秒
batch_size	64	125 个 train batch / epoch
正则	L1, 0.005	norm_weight
阈值	0.5	概率 → 0/1
seed	42	初始化 + DataLoader

10.2 五步循环

代码 10-1 内层循环

```
for xb, yb in train_loader:
    opt.zero_grad()          # 1 清空梯度
    p = model(xb)            # 2 前向
    loss = criterion(p, yb) + 0.005 * model.normnize()  # 3 损失
    loss.backward()          # 4 反向
    opt.step()               # 5 更新
```

每个 epoch 后再跑验证集：model.eval() + torch.no_grad()，算 loss 和 accuracy。先跑 5 个 epoch 做冒烟测试，确认 loss 下降，再跑 100。

10.3 模板里的三个小问题

表 10-2 模板写法问题

问题	现象	建议
先验证后训练	日志顺序混乱，不影响梯度	改成先训练后验证
val_input / val_label 覆盖外层变量	变量名遮蔽，排查困难	内层用 xb / yb
每个 batch 新建 loss_fn	额外开销，风格不统一	在 criterion 内创建一次即可

冒烟测试

5 epochs 后验证 Accuracy 约 0.63，AUC 约 0.69；如果 loss 不降或 NaN，先检查形状、dtype、clamp，不要急着调学习率。

- 先看 5 个 epoch 的 loss 趋势，再决定要不要跑满 100。
- train 与 val 是否同向下降？只有 train 降，要警惕过拟合。
- 把 batch 数、形状、seed 记进 worklog——这是复现的三件套。

日志至少留下

前 5 个 epoch 的 train/val loss、最终三项指标、seed 与启动命令。没有日志的“跑通”不可复现。

11 评估与结果

0.635 算不算好?

11.1 指标定义

表 11-1 指标

指标	含义	本实验（test）
Accuracy	预测正确的比例	0.6350
Precision	预测为 1 的样本中真正为 1 的比例	0.6278
Recall	真实为 1 的样本中被找回的比例	0.6265
F1	Precision 与 Recall 的调和平均	0.6272
ROC-AUC	排序质量，与阈值无关	0.6897

五个指标的定义：

$$Accuracy = (TP + TN) / N, \quad Precision = TP / (TP + FP)$$

式 11-1 Accuracy / Precision

$$Recall = TP / (TP + FN), \quad F1 = 2PR / (P + R)$$

式 11-2 Recall / F1

11.2 混淆矩阵

表 11-2 测试集混淆矩阵（阈值 0.5）

	Pred 0	Pred 1
True 0	328 (TN)	182 (FP)
True 1	183 (FN)	307 (TP)

本测试集：准确率 0.6350（635/1000）；精确率 0.6278（307/489）；召回率 0.6265（307/490）；特异度 0.6431（328/510）。

11.3 结果表

表 11-3 本实验结果

数据	Accuracy	Precision	Recall	F1	ROC-AUC
train (8000)	0.6446	0.6453	0.6442	0.6448	0.7005
val (1000)	0.6550	0.6606	0.6515	0.6560	0.6969
test (1000)	0.6350	0.6278	0.6265	0.6272	0.6897

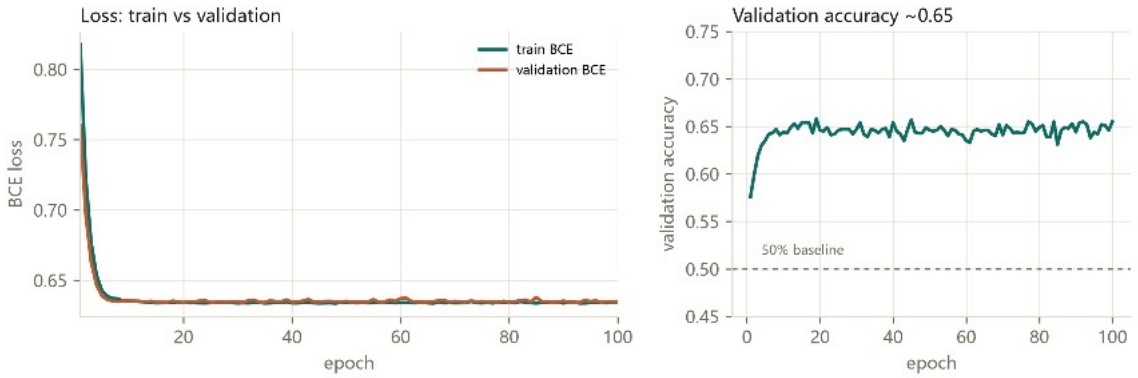


图 11-1 训练/验证损失与验证准确率；约 15–20 epoch 收敛。

11.4 怎么读这三张图

- train 与 val 的 BCE 几乎重合，没有明显过拟合。
- val accuracy 在 0.65 附近波动；最佳 epoch 19 的 0.658 与最终 0.655 只差 3 条样本。
- AUC 0.69 说明特征有中等区分能力；更强的树模型也只到约 0.695，瓶颈更像特征信息量。

> **0.635 算不算好**
>
> 50% 是基线；1000 条测试的准确率标准误约 1.5pt；sklearn 线性基线 AUC 0.692、HGB 0.695。0.635 比基线好，但不要包装成“效果很好”。

11.5 不要过度解释权重

L1 把 34/44 个权重压到 0，剩下的主要是 grade、interestRate、term、ficoRangeHigh、homeOwnership。grade 与 interestRate 高度共线，L1 只保留其中一个；权重方向可以解释，但不能当因果结论。policyCode 是常量，权重恒 0 不代表它“不重要”。

12 环境、提交与评分

三条命令、一个命名、一张对照表

12.1 环境

代码 12-1 手册推荐环境

```
conda create -n logic python=3.9 -y
conda activate logic
pip install torch==2.6.0 numpy==1.26.4 pandas scikit-learn jupyter tqdm
```

本次实际验证环境是 Python 3.11.9 + torch 2.14.1+cpu + numpy 1.26.4 + pandas 2.2.2 + scikit-learn 1.5.1；手册明确说版本不严格，能 import 即可。

12.2 VSCode

01 安装 Python / Jupyter 插件，打开包含 notebook、utils.py、data/ 的目录。

02 Ctrl+Shift+P → Python: Select Interpreter，选择 logic 环境或 .venv。

03 打开 logic.ipynb，右上角选择 kernel，点运行；先跑 import torch 验证。

12.3 提交命名

> **手册 §1.5.4**
>
> 请将 logic.ipynb 重命名为 学号_姓名_logic.zip，提交至 BB 系统。注意文件名是小写 logic.ipynb，模板是 Logic.ipynb；Windows 不区分大小写，Linux/BB 可能区分。

代码 12-2 生成提交包（本工作区提供）

```
cd submission
..\.venv\Scripts\python.exe .\pack_submission.py --student-id 你的学号 --name 你的姓名
```

12.4 评分项 ↔ 代码单元

表 12-1 10 分细则与代码位置

评分项	分值	代码位置	证据
环境	0.5	conda / .venv / VSCode	import torch 成功
读取三集	0.5	cell 2	三个 shape
清洗与缺失	1.0	cell 2 / 4	0 NaN、日期/grade 转换
Dataset/DataLoader	1.0	cell 8 / 10	125/16/16 batch

评分项	分值	代码位置	证据
模型初始化	1.0	cell 13	w (44,1)、b (1,)
sigmoid	1.0	cell 13	输出 (0,1)
forward	1.0	cell 13	(N,1) 概率
L1/L2	1.0	cell 13	normnize 可切换
损失	1.0	cell 13	BCELoss + flatten + clamp
测试评估	1.0	cell 15	Accuracy 等指标
markdown 反思	1.0	最后一格	收获/难度/建议

12.5 提交前检查清单

- 从头运行 notebook，0 error；所有代码单元都有执行序号。
- 最后一格是 markdown，包含收获、难度、建议。
- 输出目录有 metrics.json、test_predictions.csv 或等价证据。
- 提交文件名为 学号_姓名_logic.zip；logic.ipynb 小写。
- worklog 记录了 seed、命令、指标和至少一个真实问题。

13 Agent、Prompt、Harness：背景与原理

从经典 Agent 到可验收的循环

这一节回答三个问题：Agent 为什么比 LLM 古老？Prompt、Context、Harness 各管什么？同一个模型，为什么换一套 Harness 就可能从“会聊天”变成“能交付”？

13.1 经典 Agent：感知—决策—行动

AIMA 第 2 章把 agent 放在环境里定义：传感器感知环境，执行器作用于环境，理性 agent 选择能最大化性能度量的行动。Wooldridge & Jennings（1995）则给出软件工程版本：自主、交互、感知、主动。它们都不要求神经网络，也不要求自然语言。

表 13-1 经典 Agent 与 coding agent 的对应

经典 Agent	Coding Agent
传感器：摄像头 / 温度计	read / grep / 终端输出 / 测试报错
决策：规则 / 规划器 / 策略	LLM 读上下文，选择下一个 tool call
执行器：电机 / 阀门	编辑文件、跑 python、执行 git
环境状态：房间 / 温度	工作区文件、notebook 输出、metrics.json
反馈：观察结果、调整	报错、断言失败、指标变化 → 下一轮

ReAct（2022）把“推理”和“行动”交错起来：想一步、做一步、看结果，再想下一步。今天多数 coding agent 的骨架仍然是：观察 → 决策 → 工具 → 反馈。

> **两个关键区分**
>
> ① Workflow 的路径由代码预先写死；Agent 的下一步由模型动态决定。② 模型只输出 tool call，真正执行工具、回填结果的是 Harness。

13.2 Prompt Engineering：把任务说到可执行

表 13-2 Prompt 的五段结构

要素	问什么	本实验示例
角色	你是谁?	课程助教，熟悉 PyTorch 与手册 §1.6
任务	要完成什么可验收的目标?	补全 notebook 的 TODO，使其从头可执行
上下文	需要哪些材料?	手册、data/、utils.py、报错片段
约束	什么不能做?	不改数据划分；不调测试集；seed=42
输出 / 自检	交付什么、怎么验?	改动摘要 + 命令 + 指标 + verify 结果

常用技巧：给 1–3 个 few-shot 示例；把任务拆成可单独验证的小步；显式写出禁止事项；要求结构化输出；先让模型复述验收标准再动手。Prompt 的局限是：拿不到新信息、不能保证执行、不能跨轮持久、不能自证正确——这些要靠 Context 与 Harness 补。

13.3 Context Engineering：有限的注意力预算

Context 是一次采样中进入模型的所有 token。上下文越长，模型对其中信息的准确召回越弱（context rot）；所以好的 context 不是“更多”，而是“最小的高信号集合”。

表 13-3 三种 context 策略

策略	做法	适用
Compaction	把长历史压缩成摘要，保留决策与未解决问题	长对话、连续任务
Notes	把进度写到文件 / to-do，跨轮恢复	有里程碑的长任务
Sub-agents	独立上下文做深搜，只回传摘要	并行探索、海量材料

13.4 Harness Engineering：让循环跑起来

表 13-4 Harness 七件套

部件	职责	本实验中的形态
Agent Loop	模型调用 → 工具执行 → 结果回填	改 notebook、运行、看报错、再改
Context	决定每一步放进窗口的内容	手册摘录、utils.py、报错、历史压缩
Tool Runtime	提供可执行工具	读写文件、跑 python、grep、后台任务
Sandbox & Permissions	限制范围、管理授权	只写工作区；改数据前询问
Session & Memory	维护会话与跨步连续性	session 日志、WORKLOG
Orchestration	子任务、子 agent、多阶段	一个读手册、一个跑实验、主 agent 汇总
Logging & Evaluation	记录过程、提供验收	run.log、metrics.json、43 项自检

表 13-5 同一模型的三档 Harness

	A · 纯聊天	B · 有工具	C · 完整 Harness
能看到	一段文本	整个工作区	工作区 + 手册 + 历史 + 报错
能做什么	只能回答	改文件、跑命令	同 B，且有权限与追溯
怎么算完成	你说了算	它说完成	测试、自检、日志对上

SWE-agent（2024）的实证说明：为 agent 专门设计的 agent-computer interface（ACI）会显著影响它编辑代码、浏览仓库、执行测试的表现。接口不是装饰，是能力的一部分。

13.5 三层分工与 AGENTS.md

表 13-6 三层分工

层次	回答的问题	本实验
Prompt	怎么说清楚?	补全 TODO，不许改数据划分
Context	这一步该看到什么?	手册 §1.6、utils.py、报错片段
Harness	能做什么、怎么算完成?	工具、权限、43 项自检、日志

代码 13-1 10 行 AGENTS.md 示例

```
# AGENTS.md
## 项目
逻辑回归实验：notebook + utils.py + data/。
## 命令
- 训练：python run_experiment.py
- 自检：python verify_submission.py
## 边界
- 只读 data/ 与 source_materials/；只写 outputs/ 与 logreg_lab/。
- 不改 utils.py 的预处理；不改数据划分；seed 固定 42。
- 任何“已完成”必须附命令、指标和文件路径。
```

表 13-7 推荐目录边界

路径	权限	说明
data/	只读	原始 CSV；不要覆盖
source_materials/	只读	原手册、模板、任务 PPT
logreg_lab/	可写	notebook、脚本、输出
outputs/	可写	metrics、日志、图
utils.py	不改	课程给定；要改先在 worklog 说明

文件不是机制

AGENTS.md / GOAL.md 只是上下文与指令的载体；Harness 决定何时读、读了算不算数。DSH 的 /goal 是运行时状态，和任何 .md 文件都不是一回事。

动手

写一份 10 行 AGENTS.md，让 Agent 先复述：目标、输入、禁止事项、验收命令。复述不对，不要让它动手。

14 用 Agent 定位与验证 bug

先要最小复现，再要修复

Agent 最擅长的是“读报错、查文件、给候选修复”；最容易出错的是“没复现就说修好了”。下面五个坑来自本实验的真实调试记录，适合作为 Agent 协作练习。

表 14-1 五个真实 bug

编号	现象	让 Agent 做什么	你必须验证什么
D1	loss 不收敛或数值异常	打印 p/y 的 shape，解释 BCELoss 广播	loss 是标量，且形状为 (N,)
D2	训练到一半 NaN/Inf	复现 sigmoid 饱和，给出 clamp 方案	100 epoch 无 NaN
D3	notebook 与脚本同 seed 结果不同	对比验证循环次数，定位 RNG 消耗	独立 generator 后指标一致
D5	日期特征分布看起来偏移	解释按 split 取最小值的差异	train-only 基准的对照实验
D9	以为 1000 条要跑 8 个整批	打印 batch 数与末批 shape	len(loader)=16，末批 40

14.1 提问模板

代码 14-1 定位 bug 的 prompt 模板

```
现象：<报错/指标/截图>
最小复现：<命令 + 数据 + seed>
请先解释原因，不要改代码。
给出一个最小修复 diff，并说明如何验证。
最后列出你仍不确定的两点。
```

14.2 案例：RNG 耦合

脚本每 epoch 调 predict_proba 和 evaluate_loss 各遍历一次 val_loader；notebook 只遍历一次。DataLoader 即使 shuffle=False，每次 iter() 也会从全局 RNG 抽一个 base seed。脚本多遍历一次，就多消耗一次 RNG，后续 train_loader 的打乱顺序不同，指标出现 0.001 级差异。

代码 14-2 修复：独立 generator

```
generator = torch.Generator().manual_seed(42)
train_loader = DataLoader(..., shuffle=True, generator=generator)
```

验证方式：notebook 与脚本各跑一次，比较 metrics.json 的 accuracy / AUC / 混淆矩阵，要求逐位一致；再换 seed=43 重复，确认差异来自随机性而非代码路径。

> **证据链**
>
> 现象（截图/日志）→ 最小复现（命令）→ 根因（代码行）→ 修复（diff）→ 验证（指标/测试）。缺任何一环，都只能算“猜测”。

15 用 Agent 做记录与验收

把 10 分细则变成可勾选的证据

Agent 可以帮你生成 ACCEPTANCE 草稿、汇总指标、写 worklog 骨架；但“是否完成”必须由证据决定。这一节给一个可复用的验收表。

15.1 评分项 → 证据

表 15-1 验收证据表

评分项	证据行	在哪看
环境	python -V、import torch、pip list	终端 / requirements.txt
读取三集	三个 shape	notebook cell 2 输出
清洗缺失	0 NaN、行数不变	cell 4 输出
Dataset/DataLoader	125/16/16 batch	cell 10 输出
模型	w/b shape、forward shape、normnize	cell 13 输出
损失	BCE 标量、无 NaN	训练日志
测试评估	Accuracy/AUC/混淆矩阵	cell 15 输出 / metrics.json
反思	最后一格 markdown	notebook 末尾

15.2 ACCEPTANCE.md 模板

代码 15-1 ACCEPTANCE 草稿

```
# 验收记录
| 项目 | 证据 | 状态 |
|---|---|---|
| 环境 | import torch 2.14.1 | PASS |
| 读取 | (8000,47) (1000,47) (1000,47) | PASS |
| 缺失 | 0 NaN | PASS |
| 模型 | w(44,1), b(1,) | PASS |
| 评估 | Acc 0.6350, AUC 0.6897 | PASS |
| 反思 | 最后一格 markdown | PASS |
```

15.3 自动化自检

本工作区的 verify_submission.py 检查 43 项：文件、notebook 执行、TODO、指标一致性、数据 SHA-256、模型可加载、最后一格反思、提交版同步等。运行：

代码 15-2 自检命令

```
cd logreg_lab
python verify_submission.py
```

15.4 WORKLOG 应该写什么

- 材料与任务：数据从哪来、多少行、标签是什么。
- 方法与配置：预处理、模型、损失、优化器、seed。
- 主要结果：train/val/test 指标、混淆矩阵、图。
- 问题与修复：至少一个真实 bug，含现象、定位、修复、验证。
- 局限与改进：数据、方法、评估各一条。

> **Agent 自述不是证据**
>
> “已完成”是文本；测试通过、指标可复算、文件存在才是事实。让 Agent 给出命令和路径，自己再跑一遍。

16 边界、引用与学术诚信

哪些可以交给 Agent，哪些必须自己来

16.1 三条红线

01 方法学：data_utils 对每个 split 各自求日期基准与均值；严格做法是 train-only。照抄能拿分，但报告里要指出差异。

02 数字巧合：验证集 0.6550 恰好等于手册示例 0.655，这只是吻合，不是正确性证明；反过来，数字不同也不代表错。

03 反思：手册要求的“收获、难度、建议”必须用自己的话写；Agent 可以帮你整理，但不能替你经历。

16.2 Agent 使用声明

代码 16-1 使用声明模板

```
## Agent 使用声明
- 使用工具：DeepSeek Harness / 其他（写明）
- 用途：读手册、改 notebook、跑测试、整理 worklog
- 未使用：数据划分、测试集调参、代替本人写反思
- 人工复核：指标口径、混淆矩阵、提交命名、结论
- 证据：run.log / metrics.json / verify_submission.py 输出
```

16.3 引用与事实

引用网页时保留标题、机构/作者、年份、URL、访问日期。区分“文献事实”与“教学类比”：例如“能力乘积公式”是本手册的教学近似，不是论文定律；“扫地机器人”是类比，不是实验结论。

表 16-1 引用类型

类型	写法	本手册例子
事实	作者/机构，年份，标题，URL	ReAct, arXiv:2210.03629, 2022
类比	明确标注“教学类比”	Agent 是司机，Harness 是车
课程内部	手册 §x / 工作区文件	逻辑回归-实验手册 §1.6

最后一句

Agent 可以提出方案，但不能替你判断结果是否可信。把“它说完成了”换成“证据在哪”，这是本课程真正想训练的能力。

附录 A 参考资料

访问日期 2026-10-28

[01] Russell, S., & Norvig, P. Artificial Intelligence: A Modern Approach, 4th ed., Ch. 2 Intelligent Agents.

[02] Wooldridge, M., & Jennings, N. R. (1995). Intelligent Agents: Theory and Practice. Knowledge Engineering Review, 10(2).

[03] Anthropic (2024). Building effective agents. anthropic.com/engineering/building-effective-agents

[04] Anthropic (2025). Effective context engineering for AI agents. anthropic.com/research/effective-context-engineering-for-ai-agents

[05] Yao, S. et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.

[06] Yang, J. et al. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793.

[07] OpenAI (2026). Harness engineering. openai.com/index/harness-engineering/（页面 403；仅核标题/年份）

[08] DeepSeek (2026). DeepSeek Harness 官方介绍. https://www.deepseek.com/zh/harness/

[09] DeepSeek Harness 架构文档：core subsystems（agent-loop / session / tools / system-prompt）.

[10] AGENTS.md: a simple, open format for guiding coding agents. https://agents.md/

[11] Schulhoff et al. (2024). The Prompt Report. arXiv:2406.06608.

[12] 本课程：《逻辑回归-实验手册》§1.6；logreg_lab/WORKLOG.md、ACCEPTANCE.md.

附录 B 命令、数字与 Prompt 速查

打印这一页贴在电脑旁

B.1 命令

代码 B-1 常用命令

```
cd logreg_lab
python run_experiment.py
python run_experiment.py --ablation
python verify_submission.py
cd ../submission
..\.venv\Scripts\python.exe .\pack_submission.py --student-id 学号 --name 姓名
```

B.2 数字

数字	值
原始列 / 入模维	47 / 44
train / val / test	8000 / 1000 / 1000
训练集正样本	4005 / 8000 = 50.06%
缺失单元（train）	5595（19 列）
batch 数	125 / 16 / 16（末批 40）
参数量	45
验证 / 测试 Accuracy	0.6550 / 0.6350
测试 ROC-AUC	0.6897

B.3 三份模板

代码 B-2 三句话模板

```
任务书：目标 + 输入 + 约束 + 输出 + 验收。
定位 bug：现象 + 最小复现 + 根因 + 修复 diff + 验证。
验收：10 分细则 → 每项一行证据 → 命令/文件路径。
```

> **最后的建议**
>
> 先自己跑通，再让 Agent 帮你查漏；先写证据，再写结论；先把 worklog 写清楚，再提交。这三件事，Agent 都不能替你做。